

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Казанский национальный исследовательский технический университет им.
А.Н. Туполева – КАИ»

Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной Безопасности

Лабораторная работа № 1

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Работу выполнил

Студент гр. 4404

Романов Е.М.

Проверил

Шигабутдинов М.И.

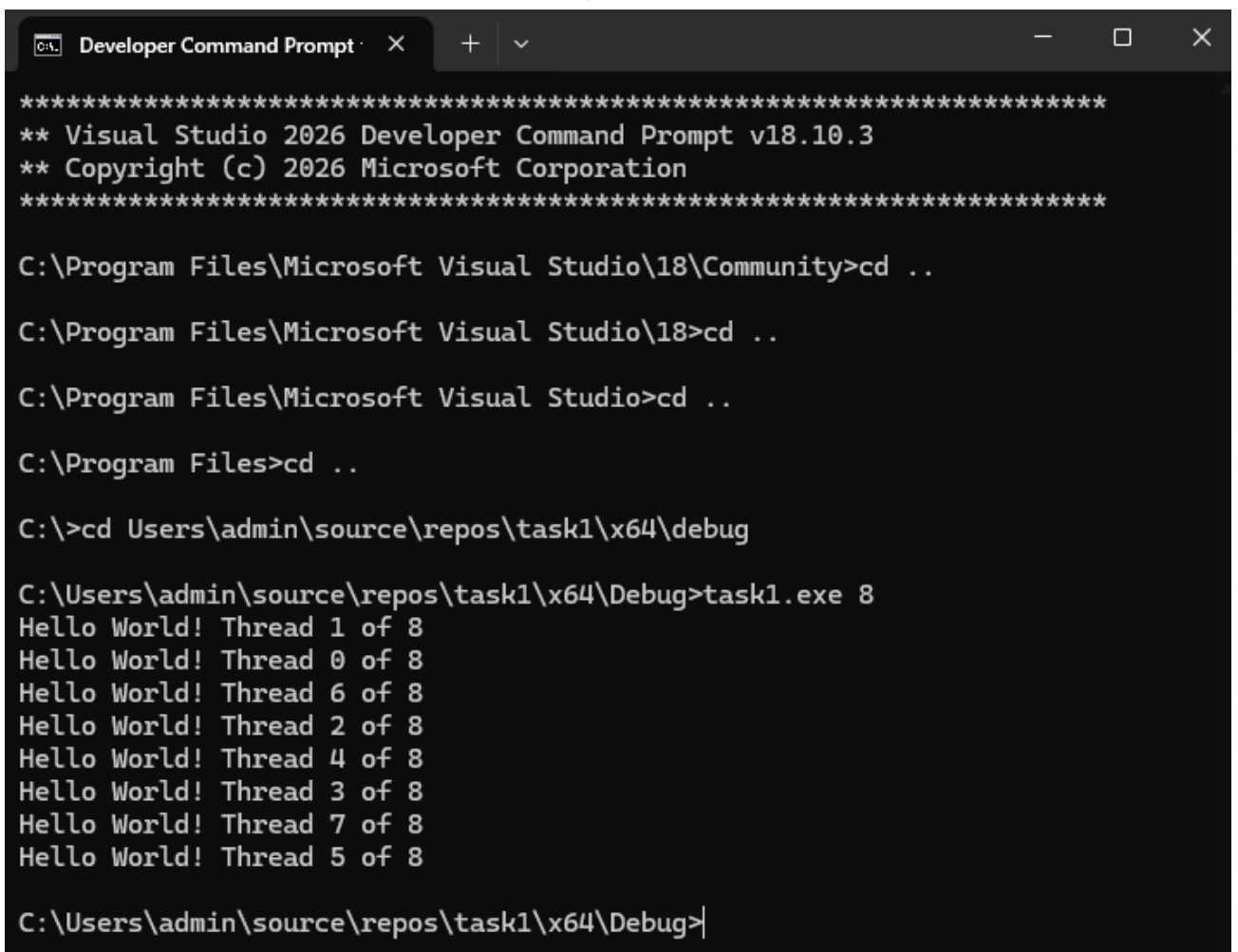
Казань, 2026

Задание 1

Написать программу где каждый поток печатает свой идентификатор, количество потоков всего и строчку «Hello World». Запустить программу с 8 потоками. Всегда ли вывод идентичен? Почему?

Программа принимает из командной строки один аргумент - число потоков. С помощью `omp_set_num_threads()` задаётся количество потоков. Затем в параллельной области (`#pragma omp parallel`) каждый поток получает свой идентификатор через `omp_get_thread_num()` и общее число потоков через `omp_get_num_threads()`, после чего печатает строку «Hello World! Thread X of Y». Число потоков передаётся через командную строку.

Запуск 1

A screenshot of a Windows Developer Command Prompt window. The title bar shows 'C:\. Developer Command Prompt'. The window contains the following text:

```
*****  
** Visual Studio 2026 Developer Command Prompt v18.10.3  
** Copyright (c) 2026 Microsoft Corporation  
*****  
  
C:\Program Files\Microsoft Visual Studio\18\Community>cd ..  
  
C:\Program Files\Microsoft Visual Studio\18>cd ..  
  
C:\Program Files\Microsoft Visual Studio>cd ..  
  
C:\Program Files>cd ..  
  
C:\>cd Users\admin\source\repos\task1\x64\debug  
  
C:\Users\admin\source\repos\task1\x64\Debug>task1.exe 8  
Hello World! Thread 1 of 8  
Hello World! Thread 0 of 8  
Hello World! Thread 6 of 8  
Hello World! Thread 2 of 8  
Hello World! Thread 4 of 8  
Hello World! Thread 3 of 8  
Hello World! Thread 7 of 8  
Hello World! Thread 5 of 8  
  
C:\Users\admin\source\repos\task1\x64\Debug>
```

Запуск 2

```
Developer Command Prompt
C:\Program Files\Microsoft Visual Studio\18>cd ..
C:\Program Files\Microsoft Visual Studio>cd ..
C:\Program Files>cd ..
C:>cd Users\admin\source\repos\task1\x64\debug
C:\Users\admin\source\repos\task1\x64\Debug>task1.exe 8
Hello World! Thread 1 of 8
Hello World! Thread 0 of 8
Hello World! Thread 6 of 8
Hello World! Thread 2 of 8
Hello World! Thread 4 of 8
Hello World! Thread 3 of 8
Hello World! Thread 7 of 8
Hello World! Thread 5 of 8

C:\Users\admin\source\repos\task1\x64\Debug>task1.exe 8
Hello World! Thread 1 of 8
Hello World! Thread 0 of 8
Hello World! Thread 4 of 8
Hello World! Thread 6 of 8
Hello World! Thread 2 of 8
Hello World! Thread 5 of 8
Hello World! Thread 3 of 8
Hello World! Thread 7 of 8

C:\Users\admin\source\repos\task1\x64\Debug>
```

Запуск 3

```
Developer Command Prompt
Hello World! Thread 1 of 8
Hello World! Thread 0 of 8
Hello World! Thread 6 of 8
Hello World! Thread 2 of 8
Hello World! Thread 4 of 8
Hello World! Thread 3 of 8
Hello World! Thread 7 of 8
Hello World! Thread 5 of 8

C:\Users\admin\source\repos\task1\x64\Debug>task1.exe 8
Hello World! Thread 1 of 8
Hello World! Thread 0 of 8
Hello World! Thread 4 of 8
Hello World! Thread 6 of 8
Hello World! Thread 2 of 8
Hello World! Thread 5 of 8
Hello World! Thread 3 of 8
Hello World! Thread 7 of 8

C:\Users\admin\source\repos\task1\x64\Debug>task1.exe 8
Hello World! Thread 0 of 8
Hello World! Thread 4 of 8
Hello World! Thread 3 of 8
Hello World! Thread 6 of 8
Hello World! Thread 2 of 8
Hello World! Thread 7 of 8
Hello World! Thread 1 of 8
Hello World! Thread 5 of 8

C:\Users\admin\source\repos\task1\x64\Debug>
```

Всегда ли вывод идентичен? Почему?

Нет, вывод не идентичен, порядок меняется от запуска к запуску. Это происходит потому, что каждый поток выполняется параллельно, планировщик не гарантирует, в каком порядке они дойдут до `printf`. Набор всегда одинаковый, порядок нет.

Вывод: порядок выполнения потоков не определён и может меняться от запуска к запуску. Для управления порядком необходимо использовать средства синхронизации.

Задание 2

Написать программу, в которой объявлен массив из 16000 элементов и инициализирован так, что значение элемента массива равно его порядковому номеру. Затем создайте результирующий массив, в котором (за исключением крайних элементов) будут средние значения исходного массива: $b[i] = (a[i-1] + a[i] + a[i+1])/3.0$. Запустите программу с 8-ю процессами при различных типах распределения работ.

Программа принимает из командной строки два аргумента: число потоков и размер массива N. Массив a инициализируется значениями $a[i] = i$. Затем параллельный цикл `#pragma omp parallel for schedule(runtime)` вычисляет $b[i]$ для $i = 1..N-2$. Крайние элементы $b[0]$ и $b[N-1]$ копируются из a. После вычислений печатаются первые 10 и последние 10 элементов массива b для проверки корректности.

Распределение static - итерации равномерно распределяются по потокам.

```
Developer Command Prompt
C:\>cd Users\admin\source\repos\task2\x64\Debug
C:\Users\admin\source\repos\task2\x64\Debug>set OMP_SCHEDULE=static
C:\Users\admin\source\repos\task2\x64\Debug>task2.exe 8 16000
First 10 elements of b:
b[0] = 0.000
b[1] = 1.000
b[2] = 2.000
b[3] = 3.000
b[4] = 4.000
b[5] = 5.000
b[6] = 6.000
b[7] = 7.000
b[8] = 8.000
b[9] = 9.000
...
Last 10 elements of b:
b[15990] = 15990.000
b[15991] = 15991.000
b[15992] = 15992.000
b[15993] = 15993.000
b[15994] = 15994.000
b[15995] = 15995.000
b[15996] = 15996.000
b[15997] = 15997.000
b[15998] = 15998.000
b[15999] = 15999.000
C:\Users\admin\source\repos\task2\x64\Debug>
```

Распределение static,1000 - блоки по 1000 итераций.

```
C:\Users\admin\source\repos\task2\x64\Debug>set OMP_SCHEDULE=static,1000

C:\Users\admin\source\repos\task2\x64\Debug>task2.exe 8 16000
First 10 elements of b:
b[0] = 0.000
b[1] = 1.000
b[2] = 2.000
b[3] = 3.000
b[4] = 4.000
b[5] = 5.000
b[6] = 6.000
b[7] = 7.000
b[8] = 8.000
b[9] = 9.000
...
Last 10 elements of b:
b[15990] = 15990.000
b[15991] = 15991.000
b[15992] = 15992.000
b[15993] = 15993.000
b[15994] = 15994.000
b[15995] = 15995.000
b[15996] = 15996.000
b[15997] = 15997.000
b[15998] = 15998.000
b[15999] = 15999.000
```

Распределение dynamic - работа распределяется пакетами заданного размера (по умолчанию размер равен 1) между потоками. Как только какой-либо из потоков заканчивает обработку своей порции данных, он захватывает следующую.

```
C:\Users\admin\source\repos\task2\x64\Debug>set OMP_SCHEDULE=dynamic

C:\Users\admin\source\repos\task2\x64\Debug>task2.exe 8 16000
First 10 elements of b:
b[0] = 0.000
b[1] = 1.000
b[2] = 2.000
b[3] = 3.000
b[4] = 4.000
b[5] = 5.000
b[6] = 6.000
b[7] = 7.000
b[8] = 8.000
b[9] = 9.000
...
Last 10 elements of b:
b[15990] = 15990.000
b[15991] = 15991.000
b[15992] = 15992.000
b[15993] = 15993.000
b[15994] = 15994.000
b[15995] = 15995.000
b[15996] = 15996.000
b[15997] = 15997.000
b[15998] = 15998.000
b[15999] = 15999.000
```

Распределение dynamic,1000 - поток берёт по 1000 итераций, закончил - берёт следующую порцию

```
C:\Users\admin\source\repos\task2\x64\Debug>set OMP_SCHEDULE=dynamic,1000

C:\Users\admin\source\repos\task2\x64\Debug>task2.exe 8 16000
First 10 elements of b:
b[0] = 0.000
b[1] = 1.000
b[2] = 2.000
b[3] = 3.000
b[4] = 4.000
b[5] = 5.000
b[6] = 6.000
b[7] = 7.000
b[8] = 8.000
b[9] = 9.000
...
Last 10 elements of b:
b[15990] = 15990.000
b[15991] = 15991.000
b[15992] = 15992.000
b[15993] = 15993.000
b[15994] = 15994.000
b[15995] = 15995.000
b[15996] = 15996.000
b[15997] = 15997.000
b[15998] = 15998.000
b[15999] = 15999.000
```

Распределение guided - размер блока уменьшается динамически (сначала большие, потом маленькие)

```
C:\Users\admin\source\repos\task2\x64\Debug>set OMP_SCHEDULE=guided

C:\Users\admin\source\repos\task2\x64\Debug>task2.exe 8 16000
First 10 elements of b:
b[0] = 0.000
b[1] = 1.000
b[2] = 2.000
b[3] = 3.000
b[4] = 4.000
b[5] = 5.000
b[6] = 6.000
b[7] = 7.000
b[8] = 8.000
b[9] = 9.000
...
Last 10 elements of b:
b[15990] = 15990.000
b[15991] = 15991.000
b[15992] = 15992.000
b[15993] = 15993.000
b[15994] = 15994.000
b[15995] = 15995.000
b[15996] = 15996.000
b[15997] = 15997.000
b[15998] = 15998.000
b[15999] = 15999.000
```

Распределение guided,1000 - размер блока уменьшается динамически (сначала большие, потом маленькие), но минимальный размер блока 1000.

```
C:\Users\admin\source\repos\task2\x64\Debug>set OMP_SCHEDULE=guided,1000

C:\Users\admin\source\repos\task2\x64\Debug>task2.exe 8 16000
First 10 elements of b:
b[0] = 0.000
b[1] = 1.000
b[2] = 2.000
b[3] = 3.000
b[4] = 4.000
b[5] = 5.000
b[6] = 6.000
b[7] = 7.000
b[8] = 8.000
b[9] = 9.000
...
Last 10 elements of b:
b[15990] = 15990.000
b[15991] = 15991.000
b[15992] = 15992.000
b[15993] = 15993.000
b[15994] = 15994.000
b[15995] = 15995.000
b[15996] = 15996.000
b[15997] = 15997.000
b[15998] = 15998.000
b[15999] = 15999.000
```

Вывод: тип распределения работ влияет только на то, как итерации цикла делятся между потоками, но не на результат вычислений. Значения $b[i]$ одинаковы для всех типов. Разница проявляется в производительности, но не в выводе, поэтому все скриншоты показывают одинаковые значения $b[0\dots9]$ и $b[15990\dots15999]$.

Задание 3

Модифицируйте задачу 1 так, чтобы потоки распечатывали свои идентификаторы в обратном порядке. Существует как минимум 5 способов решения. Постарайтесь найти как можно больше.

Способ 1 – Ordered

Все потоки выполняются параллельно, но блок внутри `#pragma omp ordered` выполняется строго по очереди — в порядке возрастания номера итерации `i`. Печатаем `total - 1 - i`, поэтому ID идут в обратном порядке: от `total-1` до 0.

```
Developer Command Prompt
*****
** Visual Studio 2026 Developer Command Prompt v18.10.3
** Copyright (c) 2026 Microsoft Corporation
*****

C:\Program Files\Microsoft Visual Studio\18\Community>cd ..

C:\Program Files\Microsoft Visual Studio\18>cd ..

C:\Program Files\Microsoft Visual Studio>cd ..

C:\Program Files>cd ..

C:\>cd Users\admin\source\repos\task3\x64\debug

C:\Users\admin\source\repos\task3\x64\Debug>task3.exe 8 ordered

Method 1: ordered
Hello World! Thread 7 of 8
Hello World! Thread 6 of 8
Hello World! Thread 5 of 8
Hello World! Thread 4 of 8
Hello World! Thread 3 of 8
Hello World! Thread 2 of 8
Hello World! Thread 1 of 8
Hello World! Thread 0 of 8

C:\Users\admin\source\repos\task3\x64\Debug>
```

Способ 2 – Active Waiting

Используется атомарный счётчик `turn`. Каждый поток крутится в цикле, атомарно читая `turn`, пока его значение не станет равным «обратному» номеру потока. Когда очередь дошла — поток печатает и атомарно увеличивает `turn`. Потоки активно ожидают, расходуя CPU.

```
C:\Users\admin\source\repos\task3\x64\Debug>task3.exe 8 atomic

Method 2: Active Waiting
Hello World! Thread 7 of 8
Hello World! Thread 6 of 8
Hello World! Thread 5 of 8
Hello World! Thread 4 of 8
Hello World! Thread 3 of 8
Hello World! Thread 2 of 8
Hello World! Thread 1 of 8
Hello World! Thread 0 of 8
```

Способ 3 – Channel Chain

Создаётся массив флагов `flags[]`. Поток `id` ждёт, пока `flags[id + 1] == 1` (то есть пока следующий поток не напечатает). Последний поток (`id = N-1`) начинает сразу. После печати поток ставит `flags[id] = 1`, разрешая печатать предыдущему. Получается цепочка: $N-1 \rightarrow N-2 \rightarrow \dots \rightarrow 0$.

```
C:\Users\admin\source\repos\task3\x64\Debug>task3.exe 6 chain

Method 3:Channel Chain
Hello World! Thread 5 of 6
Hello World! Thread 4 of 6
Hello World! Thread 3 of 6
Hello World! Thread 2 of 6
Hello World! Thread 1 of 6
Hello World! Thread 0 of 6
```

Способ 4 – Barrier + Single

Все потоки записывают свои ID в общий массив. Затем барьер (`#pragma omp barrier`) — все ждут, пока все допишут. После барьера только один поток (`#pragma omp single`) печатает массив в обратном порядке: от `total-1` до 0.

```
C:\Users\admin\source\repos\task3\x64\Debug>task3.exe 10 barrier

Method 4: Barrier + Single
Hello World! Thread 9 of 10
Hello World! Thread 8 of 10
Hello World! Thread 7 of 10
Hello World! Thread 6 of 10
Hello World! Thread 5 of 10
Hello World! Thread 4 of 10
Hello World! Thread 3 of 10
Hello World! Thread 2 of 10
Hello World! Thread 1 of 10
Hello World! Thread 0 of 10
```

Способ 5 – Critical Section

Каждый поток в цикле заходит в критическую секцию (`#pragma omp critical`) и проверяет, его ли очередь (`turn == my_turn`). Если да — печатает и увеличивает `turn`. Если нет — выходит и пробует снова. В отличие от `atomic`, `critical` использует блокировку, поток может «засыпать», ожидая освобождения секции.

```
C:\Users\admin\source\repos\task3\x64\Debug>task3.exe 7 critical

Method 5: Critical Section
Hello World! Thread 6 of 7
Hello World! Thread 5 of 7
Hello World! Thread 4 of 7
Hello World! Thread 3 of 7
Hello World! Thread 2 of 7
Hello World! Thread 1 of 7
Hello World! Thread 0 of 7
```

Вывод: все 5 способов дают одинаковый результат — идентификаторы потоков печатаются в обратном порядке: от N-1 до 0. Способы различаются используемыми средствами синхронизации:

Ordered — использует встроенный механизм упорядочивания.

Atomic и Chain — потоки активно ждут своей очереди, расходуя CPU.

Critical — использует блокировку, поток засыпает, ожидая освобождения секции.

Barrier + Single — печатает один поток, остальные только записывают ID в массив.